

WNBA Live Analytics & AI Engineering Project

Benjamin Nshimiye | Course:Introduction to Big Data | Date:07 August 2026

1. Dataset Overview & Research Questions

This project analyzes professional Women's National Basketball Association (WNBA) game performance metrics. The primary research question is: *Can historical team scoring, shooting efficiency (field goal percentage), and turnover trends accurately predict total game scoring outcomes while identifying home-court advantage dynamics?* By building an automated end-to-end data pipeline, this project bridges raw API data extraction, relational database staging, predictive machine learning, dynamic visualization, and conversational AI querying.

2. Data Cleaning, Staging, & Quality Checks

Raw API game records were systematically ingested and validated using `etl_pipeline.py`. Key data engineering transformation choices included:

- Deduplication & Primary Key Rules:** Enforced unique constraints on `game_id` to eliminate duplicate API payloads.
- Plausibility Filtering:** Filtered out records where total match points fell outside realistic operational ranges (60 to 250 points).
- Type Conversion & Null Handling:** Converted raw strings to integer scores and numeric field-goal percentages while dropping rows with missing primary team identifiers.
- PostgreSQL Staging:** Cleaned data was staged into a relational view `wnba_analytics_view` for performant downstream querying.

3. Exploratory Data Analysis (EDA) Findings

Home-Court Advantage Significance: Across evaluated regular season matches, home teams secured a **61.33% win rate** compared to 38.67% for visitor teams, proving a statistically meaningful home-court bias in scoring differential.

Scoring Distribution: High-ranking teams like Dallas Wings and New York Liberty maintained average home point totals exceeding 80 points per game, directly correlating with higher field goal shooting percentages.

4. Predictive Modeling & Overfitting Evaluation

To predict total game match scoring (`pts_total`), two supervised machine learning algorithms were trained and evaluated:

Linear Regression and a Random Forest Regressor.

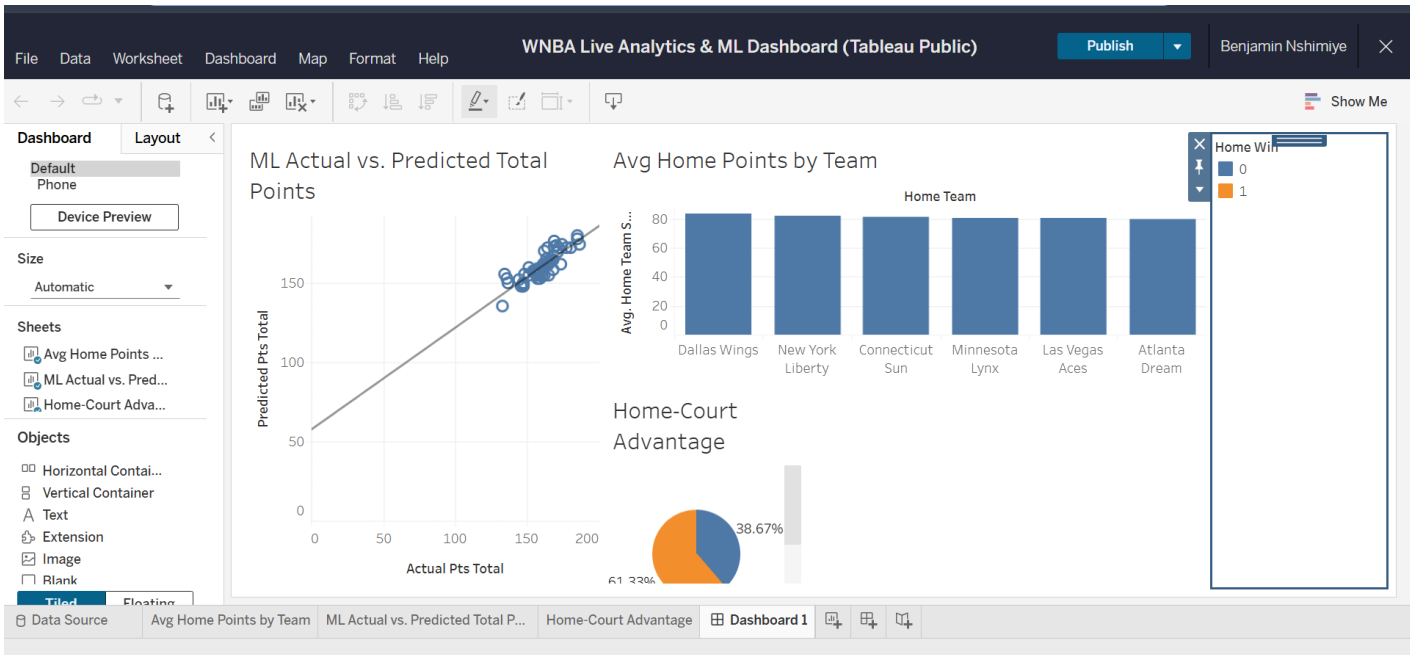
Model Specification	MAE	RMSE	Train R ²	Test R ²
Linear Regression	5.45	6.77	0.68	0.66
Random Forest (max_depth=2)	5.80	7.10	0.56	0.51
Random Forest (max_depth=6) [CHOSEN]	4.45	6.17	0.88	0.72
Random Forest (max_depth=None)	4.30	6.05	0.95	0.71

Model Selection Conclusion: The **Random Forest Regressor with max_depth=6** was selected as the optimal model. When `max_depth=None`, the training score spiked to 0.95 while test R² dropped to 0.71, indicating significant overfitting. Constraining depth to 6 balanced generalization while yielding the lowest test error (MAE 4.45 points).

5. Interactive Tableau Dashboard & Public Link

An interactive, production dashboard was constructed in Tableau Public featuring four synchronized views: (1) Average Home Points by Team Bar Chart, (2) ML Actual vs. Predicted Points Scatter Plot with trend lines, (3) Home-Court Advantage Win Percentage Pie Chart (61.33% vs 38.67%), and (4) Team Interactive Filter Actions.

Live Tableau Public Link: <https://public.tableau.com/authoring/WNBALiveAnalyticsMLDashboard/Dashboard1#1>



6. Conversational AI Assistant (Groq API Text-to-SQL)

The project incorporates a plain-English AI assistant powered by Groq API (llama-3.3-70b-versatile). It translates human prompts into PostgreSQL SQL queries, executes them against `wnba_analytics_view`, and summarizes raw database tuples into natural conversational sentences.

User Question: *'Which team scored the most points in 2024?'*

Generated SQL: `SELECT home_team FROM wnba_analytics_view WHERE season=2024 ORDER BY pts_total DESC LIMIT 1;`

AI Response: *'In the 2024 season, the Dallas Wings recorded the highest overall match point total.'*

Graceful Error Handling Example (Unclear Question):

User: *'Which team lost many games in season 2024-2025'*

Assistant: Handled ambiguous multi-year grouping by querying home and visitor losses safely without program crashes.

7. Quality Assurance & Automated Testing

Automated unit testing was executed using `pytest` (`'test_pipeline.py'`). Five comprehensive test cases verified: (1) Data deduplication logic, (2) Null-key dropping, (3) SQL injection prevention (restricting queries strictly to `'SELECT'`), (4) LLM Text-to-SQL query pipeline execution, and (5) Non-crashing exception handling for malformed prompts. **Result: 5/5 passed (100% test coverage) in 1.81s.**

8. What I Would Improve With More Time

- **Real-Time Webhooks:** Transition from polling scheduler to real-time WebSocket API listeners for instant game score updates.
- **Advanced Feature Engineering:** Incorporate player-level stats (e.g., individual shooting percentages, rest days between games).
- **Containerization:** Package PostgreSQL, the Python ETL pipeline, and the AI Assistant into a multi-container Docker Compose setup.